

Thixotrope, and how its native resources are built

@endo/thixotrope as of [endojs/endo-but-for-bots PR #1220](#), “feat(thixotrope): add durable services and installable native resources” (draft against 11m; head 387a61c, 24 September 2026; 48 commits, +12.8k/−1.9k lines). Statements were checked against the code at that head; file references are to `packages/thixotrope/` unless noted.

Thixotrope is a prototype distributed object-capability machine whose JavaScript vats are *orthogonally persistent*: objects, closures, pending promises and their listeners survive worker sleep and host restart with no application-written serialization. Operating-system resources cannot persist that way; a listening socket or a child process dies with the process that owns it. PR #1220 answers with **native resources**: an installable directory whose `durable.js` becomes a manager inside the persistent workspace heap, and whose `ephemeral.js` runs in a disposable Node process that owns every platform object. Sections 1–3 describe the machine and its persistence boundaries; sections 4–8 describe how a native resource is defined, installed, launched and recovered, with HTTP as the worked example and a checklist for writing another.

1 The machine at a glance

A single supervisor process (`thix serve STATE`) owns a private state directory, holds the engine’s exclusive lease, and composes the daemon, the peer netlayer, the workspace and the host services. Administration is an OCapN session over `control.sock` (mode 0600); the `thix` CLI and terminal views connect there and never own workspace lifetime.

Composition root `src/control/supervisor.js`; CLI `bin/thix.js`. The supervisor refuses a state directory that grants any group or other permission, or that another user owns.

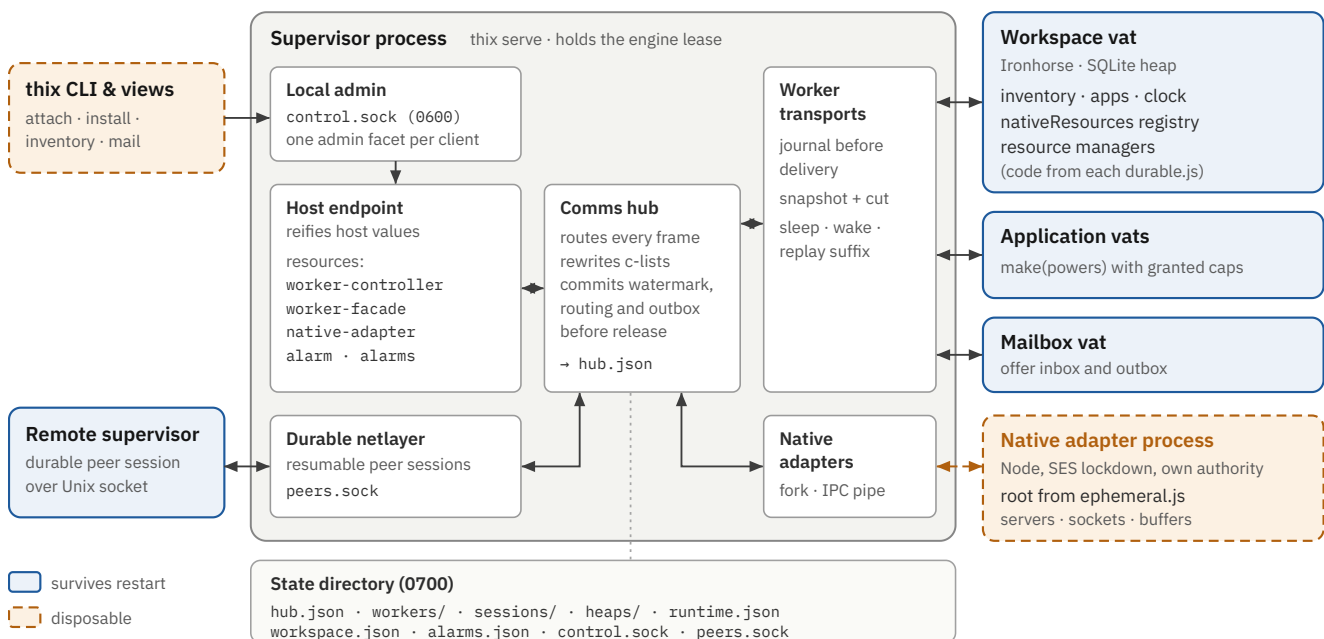


Figure 1 — Processes and sessions. Solid outlines survive restart; dashed outlines are disposable. Every guest-to-guest message crosses the hub. Vats reach it through journaled worker transports and remote supervisors through the durable netlayer. A native adapter process is simply another hub session, attached through an IPC pipe and never journaled.

Each guest vat runs in its own worker process, by default an Ironhorse machine with a SQLite heap, as an OCapN peer around one persistent SES Compartment. Guests have no ambient I/O: capabilities arrive as endowments, and every call is an eventual send.

Every message between vats, even between two local ones, crosses the **comms hub**. The hub is a forwarding node, not an OCapN client: it decodes each frame against table-backed reference kits, rewrites reference positions through per-session c-lists, and re-encodes the frame for its destination, so routed objects and promises are never materialized in the host. Its tables are written through to `hub.json` before any frame that mentions them is released.

Engines implement the `WorkerEngine` seam (`src/core/worker-engine.js`): Ironhorse by default (repo-root `rust/thixotrope-ironhorse-worker`), or XS.

Hub `src/net/hub.js`. Answers are re-exported as promise rows, so they can be listened on, pipelined and collected like any other reference.

The **host endpoint** is the deliberate exception: a session in which the host itself reifies values. It serves the system resources (the worker controller and facades, alarm promises, and the native-adapter launchers added by this PR) and the embedder's administrative route.

The **workspace** is one durable vat holding the user's state: an observable inventory Map, the application registry apps, the native-resource registry nativeResources and, once first used, the public clock and the mailbox address book. Applications are bundled modules exporting make(powers), each installed into a fresh vat with only the inventory capabilities the operator names.

Workspace globals are created idempotently, e.g. globalThis.nativeResources ??= ..., so an interrupted initialization can simply be repeated.

2 What persists, and how

Vats sleep. An idle worker finishes its step, checkpoints its heap, records the journal cut, and exits; its OCapN session and every reference into it stay live. The next frame wakes it: restore that exact image, replay the journal suffix. Sleep is host policy, invisible to guests.

Transport src/core/durable-worker-transport.js. Idle sleep is 30 s in the supervisor. Ironhorse commits each crank to SQLite before its result leaves the worker.

Delivery is journaled. Frames to a worker are journaled before the engine sees them; frames from it carry a session-lifetime sequence number, so the hub drops the duplicates that deterministic replay regenerates. The hub commits its inbound watermark, table changes and outbox frames together before releasing output, and outgoing frames keep stable delivery identities, so a repeated handoff never becomes a second invocation. The durable netlayer gives remote peers the same contract: a payload enters a durable inbox before its acceptance is acknowledged.

The host endpoint is described, not snapshotted. Each value it exports is recorded either as a *resource*, {name, description}, which a registered factory recreates at the same export slot on restart, or as *internal* (protocol plumbing, native incarnations, ephemeral observers), which is re-seated as a tombstone. Answers the host itself owes are at-most-once: one pending across a restart rejects rather than repeating an external effect.

Session records src/core/worker-session-records.js. Resource instances are interned per (name, JSON(description)), so a restored slot and a fresh request share one object.

Hub session	Key	Across a daemon restart
Worker (vat)	random 128-bit id	Restored from heap image + journal suffix, unless created ephemeral (then deleted at startup). A frame for a sleeping worker is journaled and wakes it.
Remote peer	peer:<token>, handoff:<hex>	Resumed from its session store (inbox, outbox, handshake identity); outbound handoff: sessions with pending work are redialled. A non-resumable conn:<hex> peer is not durable.
Host endpoint	endpoint	Re-seated from recorded resource descriptions; pending host-owed answers reject.
Ephemeral client	transient:<hex>	Discarded at startup. A disposable host session from daemon.openEphemeralClient().
Native adapter	transient:native:<hex>	Discarded when its process exits, and at startup. One per process incarnation; durable: false, so frames to a detached one break instead of queueing.

3 Resources that cannot be persisted

Orthogonal persistence preserves heap relationships. It does not make a socket, a process or a timer persistent, and it cannot tell whether an interrupted external operation already happened. At this head, three patterns carry capabilities across that boundary.

	Host resource	Restorable host promise	Installed native resource
Examples	worker-controller, worker-facade, native-adapter, alarms	alarm settlement (the alarm resource)	HTTP, from resources/http/
Durable record	(name, description) at the endpoint's export slot	row in alarms.json: deadline, then outcome until the guest acknowledges	desired state in the manager's workspace heap
Code runs in	the daemon	host ledger plus a guest clock in the workspace	durable.js in the workspace; ephemeral.js in its own process
After restart	factory re-runs for the same description	promise rebuilt from the ledger; the guest listener re-attaches and fires	start notice; the manager builds a fresh adapter and restores desired state
In-flight work	host-owed answers reject	settlement delivered after restart	requests dropped, never replayed

Why a separate process? Orthogonal persistence is indiscriminate. A durable vat that handled live connections would also persist connection wrappers, half-read buffers and request closures, and its author would have to decide case by case which of them may survive. Running the mechanism in a process that is *expected to die* makes “nothing here survives” structurally true: the adapter is a persistence barrier. The durable **manager** keeps policy and desired state, the disposable **adapter** keeps mechanism, and consumers hold only the manager’s facets.

Background designs/manual-persistence-vats.md (now marked historical) develops the manager/adapter vocabulary; designs/host-service-template.md counts the eleven obligations a host-side service had to hand-write.

4 Defining a native resource

A native resource is a trusted directory holding two entry modules, each exporting make(powers). The daemon contains nothing resource-specific: it supplies generic process launch, reference routing, retirement and shutdown, and the directory supplies everything else.

Contract designs/native-resource-installation.md. The generic machinery is about 460 lines in src/native/, src/adapter-keeper.js and src/platform/node/native-*.js.

	durable.js: the manager	ephemeral.js: the adapter
Runs in	the workspace vat, in its persistent heap	a fresh Node child process per incarnation
Loaded	once, at installation: bundled with compartment-mapper and evaluated into the heap	at every launch: import() ed after the directory digest is re-verified
Called as	make({E, Far, makeKeeper, adapters}), which must return synchronously	make({}), with no powers; may return a promise
Returns	{registration, lifecycle}, both remotables	one remotable, served to the daemon as root
Authority	the powers it is given; no native imports*	its own process’s ambient Node authority (e.g. node:http): SES lockdown, no Compartment
State	desired state, handles, consumer references: durable	servers, sockets, buffers, timers: die with the process
After restart	resumes exactly where it was	never restored; a successor is built and told the desired state

* The bundle is evaluated in the workspace compartment, so the powers record is the module’s intended interface, not a confinement boundary; both modules are trusted code chosen by the operator. The tree may contain only regular files and directories (no symlinks, no node_modules). Dependencies outside it are bundled into the durable half at installation, but the ephemeral half resolves them normally at every launch, unpinned.

The smallest valid resource shows the whole shape. Installation calls make once; every later daemon start calls started().

Smallest valid resource
test/fixtures/native-package/, installed under two names by test/native-install.test.js; a test-only counter and the harden(make) lines are omitted.

```
// durable.js: evaluated once, into the workspace heap
export const make = ({ Far }) => {
  let starts = 0;
  return harden({
    registration: Far('Registration', { starts: () => starts }),
    lifecycle: Far('Lifecycle', { started: () => { starts += 1; } }),
  });
};
// ephemeral.js: imported afresh by every adapter process
import { Far } from '@endo/far';
export const make = () => Far('Adapter', {});
```

registration

The public facet. Installation puts only this in the named inventory slot; applications receive it through ordinary grants.

returned by durable.js; placed by src/native/registry.js

lifecycle

The private facet, kept by the registry and never placed in the inventory. started() reconciles desired state after each daemon start.

returned by durable.js; fanned out by src/native/registry.js

adapters

The launcher: host resource native-adapter, whose description pins this directory. It is re-seated on restart, so the manager’s reference never goes stale. create() starts one incarnation.

src/native/adapters.js; minted in src/control/supervisor.js

incarnation

{getRoot(), retire()} for one process. Its references break permanently when that process exits.

src/native/adapters.js

makeKeeper

{create, restore} → {provide, retire, status}. Serializes creation and replacement, probes the current adapter, and restores desired state into a successor. It knows neither the resource kind nor the process mechanism.

src/adapter-keeper.js

root

What ephemeral.js returns: its process’s only bootstrap export, fetched through the hub under the swissnum root.

served by src/platform/node/native-worker-entry.js

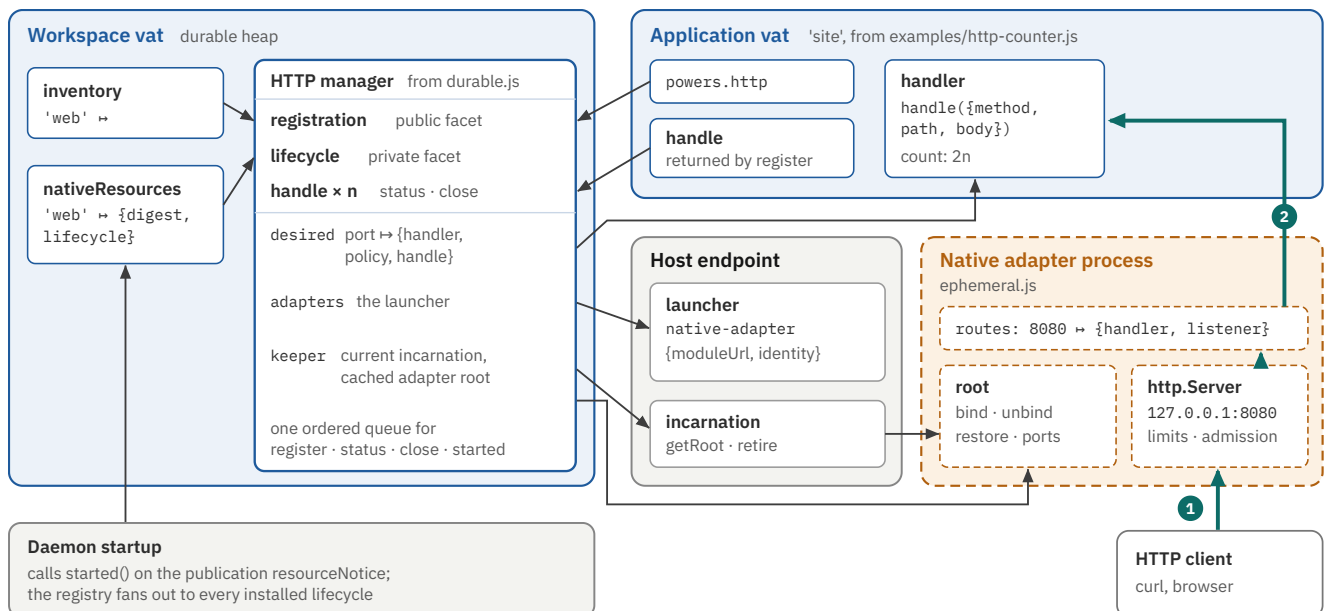


Figure 2 — An installed HTTP resource. Thin arrows are references (“holds”). The application received the registration through the grant `http=web` and holds the handle that `register` returned. While the registration is open, both the manager’s `desired` map and the adapter’s routes hold the application’s handler. Teal is a request: ① the adapter applies limits and admission and reads the body; ② it calls `E(handler).handle({method, path, body})` through the hub, and the reply retraces the path. The workspace is not on the request path.

5 Building: installation

`this install-native STATE NAME DIRECTORY` is a single control-socket call, `installNative(name, directory)`; the supervisor does the rest (Figure 3). Installations are serialized (they share one staging slot in the workspace), and supervisor shutdown waits for one it has accepted.

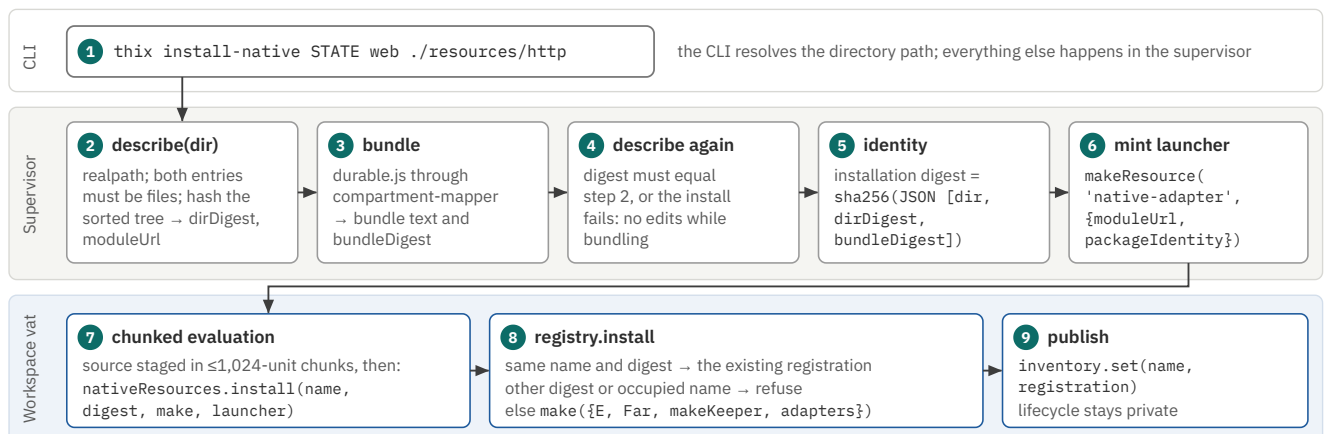


Figure 3 — Installation. The durable half is bundled and evaluated once. The ephemeral half is only hashed: nothing from `ephemeral.js` runs until an adapter is launched. The directory digest walks the tree depth-first with sorted entries; each file contributes `JSON.stringify([relativePath, byteLength])` and then its bytes.

The registry is ordinary workspace code: the supervisor ships `makeNativeResourceRegistry` and `makeAdapterKeeper` into the workspace as source text, like the inventory and application registry. Step 7 stages the source in chunks of at most 1,024 UTF-16 code units (never splitting a surrogate pair), then evaluates it. The bundle sits inside the `make` arrow, so the module’s top-level code runs only when the registry calls `make`, locally, for a new installation.

The two halves are pinned differently. The durable half is captured once: its closures live in the heap and are never re-read. The ephemeral half is re-imported by every incarnation, so the child re-hashes the directory first and refuses on any mismatch, disabling the resource rather than pairing an old manager with new native code. Reinstalling the same name and digest returns the original registration without calling `make` or touching the inventory slot. Because the installation digest covers the resolved path, the same bytes from another path count as different code. An upgrade is a new name with an

Why chunks? Sending the whole bundle in one evaluation exhausted the Ironhorse decoder’s heap ceiling. An outer destructured-parameter wrapper also hit a parser bug, fixed upstream in 242b339b8.

unchanged new directory; there is no automatic code upgrade or state migration.

6 Running: incarnations and recovery

Every adapter process is built by the same sequence, whether the trigger is the first register, the first manager call after an adapter died, or a daemon restart (Figure 4).

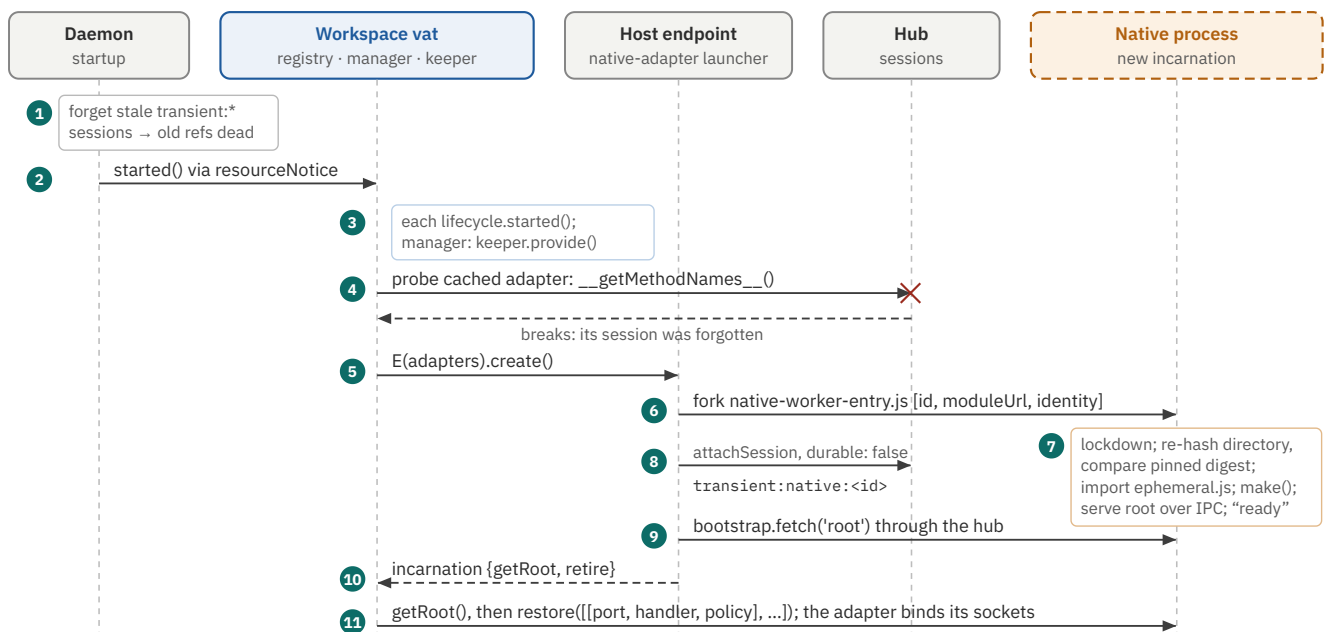


Figure 4 — Launching and restoring an incarnation. Steps 1–2 happen only at daemon startup; register, status or close enter at step 3, and step 4 runs only when an adapter is cached. If restore fails, the keeper retires the new incarnation and provide() rejects; HTTP’s register still returns its handle and status reports inactive, while close and started reject.

The launch. The child serves its root over the IPC pipe as base64 OCapN frames. Both session identities derive deterministically from a fresh random id, so there is no handshake: the pipe is the trust boundary. **Generation identity comes free:** when the process exits, the daemon calls `hub.forgetSession`, which bumps the session’s epoch, turns the rows it originated into dead tombstones, and breaks listeners waiting on it. Each incarnation has a new key, so a stale reference fails loudly and can never reach a successor.

The start notice. Waking runs none of a vat’s code, so a manager needs a delivery to learn that a new host exists. The supervisor publishes the registry’s lifecycle under a random secret kept in `workspace.json` and records it as the workspace’s `startNotify`. Each later daemon start delivers it during daemon construction (after the transient: purge and journal replay, before the control socket opens), calling `started()` and waiting at most 10 s; a rejection is logged, a timeout is not, and neither is fatal. The delivery is also the wake.

No monitor. Replacement is lazy, and the keeper never re-issues the call that discovered a death, because that call may or may not have taken effect; retrying is the caller’s decision.

Launch `src/native/adapters.js`, `src/platform/node/native-workers.js` and `native-worker-entry.js`. Fork-to-ready is bounded at 30 s (the root fetch is not); the child exits if its IPC channel closes; retirement is SIGKILL. Pipe identities come from `src/net/pipe-network.js`.

Start notice `setStartNotice` and the startup loop in `src/core/daemon.js`. The 10 s bound (`START_NOTICE_MS`) covers only `started()`. All installed resources share this one notice, and the registry passes on only its first rejection.

Event	Outcome at this head
Adapter process exits	Its session is forgotten and its references break; the resource serves nothing until the next manager operation or daemon start rebuilds it.
Supervisor SIGKILLED	Children exit when their IPC channel closes; restart purges transient:* sessions and the start notice rebuilds listeners. In-flight requests are lost, never replayed; effects already accepted by vats remain.
Port bind fails	<code>register</code> still returns a handle; <code>status()</code> reports inactive with the error and retries; other ports are restored.
<code>close()</code> interrupted	The manager retires the whole adapter through its keeper, so the withdrawn port cannot stay served; the rest are rebuilt on next use.
Directory edited	The next launch refuses to import, and <code>status()</code> reports inactive (“exited before readiness”) until the directory is restored byte for byte. Installing the edit under a new name creates a separate, empty resource.

7 Worked example: HTTP

```
thix install-native ./private-state web ./resources/http # registration → inventory 'web'
thix install ./private-state site ./examples/http-counter.js http=web # grant it to an application
# in `thix attach ./private-state`: E(E(apps).get('site')).start(8080)
curl -X POST http://127.0.0.1:8080/incr
```

The **manager** (`durable.js`, no native imports) keeps desired: `port ↦ {handler, policy, handle}.register(port, handler, policy?)` checks the port (1024–65535), the handler (remotable) and the origin list, then returns a handle with `status()` and `close()`. Operations share one ordered queue, and a handle acts only while it is still its port’s current entry, so a stale handle cannot close a later registration. Its whole connection to the native side is the keeper:

```
const keeper = makeKeeper({
  create: async () => {
    const incarnation = await E(adapters).create();
    return harden({ adapter: await E(incarnation).getRoot(),
                    retire: () => E(incarnation).retire() });
  },
  restore: adapter => E(adapter).restore(harden([...desired].map(
    ([port, { handler, policy }]) => harden([port, handler, policy])))),
});
// lifecycle.started(): if (desired.size > 0) await keeper.provide();
```

Files `resources/http/durable.js` (135 lines), `ephemeral.js` (117) and `listener.js` (225). The daemon imports none of them.

The **adapter** (`ephemeral.js` with `listener.js`) owns one Node http server per port on 127.0.0.1 and enforces the profile itself: headers ≤16 KiB, a UTF-8 text body ≤64 KiB (else 413 or 400), at most 16 concurrent requests (503), a 5 s deadline that also covers the guest call (504), and a reply ≤64 KiB (else 500). Host must equal 127.0.0.1:<port>; an Origin, if sent, must be allow-listed (default: same origin); and Sec-Fetch-Site, if sent, must be same-origin or none; otherwise 403. Only `{method, path, body}` crosses into the guest. Each incarnation shares one transient session across all of its requests; there is no host-side HTTP observer.

Admission reads only the request line and headers, before any body. Because it may be asynchronous, the request cap and deadline apply first, so an unadmitted request is still counted and timed.

8 Writing another native resource

HTTP is the only installed resource at this head, so this is the loader’s contract plus HTTP’s conventions; the design notes leave each future resource to specify its own operation outcomes and restoration policy.

Required at this head

- The directory holds `durable.js` and `ephemeral.js` as regular files, and nothing but files and directories: no symlinks, no `node_modules`.
- `durable.js` exports a synchronous `make({E, Far, makeKeeper, adapters})` returning remotable registration and lifecycle facets.
- `lifecycle.started()` is called at each later daemon start; failures are not fatal.
- `ephemeral.js` exports `make`, whose result must be (or resolve to) a remotable root; otherwise the child exits with an error.
- The directory must stay byte-identical, because every launch re-verifies its digest.

Conventions established by HTTP

- Keep desired state in the manager and hand all of it to `restore`, applied entry by entry; the keeper ignores the result, so HTTP surfaces a failed port only through `status()`.
- Run every manager operation on one ordered queue; the keeper already serializes creation and replacement.
- Return a handle even when the native operation fails, and let `status()` retry.
- Treat an interrupted operation as uncertain: HTTP retires the whole adapter rather than guess whether unbind happened.
- Keep per-event work (limits, admission) in the adapter, with policy passed at bind time, so events never wake the manager; give consumers only the manager’s facets.

LIMITS AND OPEN QUESTIONS AT THIS HEAD

- One workspace per state directory. HTTP is loopback-only with bounded text bodies, and its origin checks do not authenticate local processes.
- No autonomous restart monitor, code upgrade or state migration for installed resources; the ephemeral half’s dependencies outside the directory are unpinning.
- Workspace metadata version 3 is required; older state is rejected before any worker is restored.
- The hub calls `onAbort` only for remote sessions, and the daemon gives it no error logger, so a malformed adapter frame is dropped silently. The session is retired on process exit, explicit retirement, daemon shutdown, or the next startup’s transient: purge.